

Comparing Gradient Boosting Machines and XGBoost: First-Order vs Second-Order Optimization

Jack Neary

Introduction

High blood pressure, hypertension, is defined as having a resting systolic blood pressure (SBP) ≥ 130 mmHg, or a resting diastolic blood pressure (DBP) ≥ 80 mmHg. Hypertension can be very dangerous and often does not present for many years, while damaging the circulatory system of the body. People with hypertension are at increased risk for heart attack and stroke, both of which are leading causes of death world-wide (Di Cesare M., 2024).

Prediction of systolic blood pressure from patient health indicators can help with earlier intervention. Machine learning methods are increasingly applied to these types of prediction tasks, but the choice of algorithm and its underlying optimization can meaningfully impact predictive performance, computational efficiency at scale, and complexity trade-off.

This project evaluates two tree-based methods, Gradient Boosting Machines (GBM) and XGBoost, on a regression task to predict SBP, focusing on their underlying optimization strategies. GBM only uses first order optimization, using the gradient (first derivative) of the loss function for tree creation. XGBoost extends this objective by using the second order optimization, using both the gradient and the Hessian (second derivative), enabling a more optimal and precise leaf weight calculation and regularization than GBM.

The Cardiovascular Disease Dataset was obtained from Kaggle (Sulianova, 2019), and contains 70,000 patient records with health indicators including age, height, weight, cholesterol, glucose, smoking status, alcohol use, and physical activity level. The target variable was systolic blood pressure (ap_hi), a continuous outcome. Rows with physiologically implausible values (systolic BP < 60 or > 250 mmHg) were removed, producing 68,783 cleaned records. Data were divided into 80% for training and 20% for testing sets.

Dataset

Characteristic	Training Set (n=55,026)	Testing Set (n=13,757)
Systolic BP mean (SD), mmHg	128.8 (17.4)	128.9 (17.3)
Age, mean (SD), years	53.3 (6.8)	53.3 (6.8)
BMI, mean (SD), kg/m ²	26.4 (4.4)	26.4 (4.4)
Female, n (%)	34,789 (63.2%)	8,701 (63.3%)
Smoker, n (%)	5,652 (10.3%)	1,411 (10.3%)
Cholesterol above normal, n (%)	13,024 (23.7%)	3,254 (23.7%)

GBM

For a regression with n observations, GBM minimizes the objective function, which consists of the sum of individual losses:

$$L = \sum l(y_i, \hat{y}_i)$$

For regression with squared error loss, the individual loss for each sample is:

$$l(y_i, \hat{y}_i) = \frac{1}{2}(y_i - \hat{y}_i)^2$$

At each iteration of t , GBM computes the pseudo-residuals for each sample and fits a new shallow tree to predict these residuals. The leaf values of each tree are set to the mean of the residuals contained in that leaf. The new tree is then added to the running prediction and is scaled by a learning rate η :

$$F_t(x) = F_{t-1}(x) + \eta \cdot f_t(x)$$

GBM's objective function contains no regularization term, overfitting is controlled only through hyperparameters such as maximum tree depth, subsampling rate, and the learning rate. GBM does not have a mathematical penalty for model complexity within the objective function itself.

This model was implemented using Python's scikit-learn GradientBoostingRegressor with $n_estimators=200$, $learning_rate=0.1$, $max_depth=4$, and $subsample=0.8$.

XGBoost

XGBoost objective function builds on GBM's same loss function, and includes the regularization term Ω penalizing model complexity with λ (lambda) and γ (gamma):

$$L = \sum l(y_i, \hat{y}_i) + \sum \Omega(f_k) \quad \text{where} \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

T is the number of leaves, w are the leaf weights, γ is the minimum gain threshold for a split, and λ is the L2 regularization parameter. At each iteration of t , XGBoost approximates the objective using a second-order Taylor expansion:

$$L^{(t)} \approx \sum [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

where $g_i = \partial l / \partial \hat{y}_i$ is the gradient and $h_i = \partial^2 l / \partial \hat{y}_i^2$ is the Hessian for i . For squared error regression, $g_i = \hat{y}_i - y_i$ (the residual) and $h_i = 1$ for all samples. By minimizing this expression with respect to the leaf weights, the optimal leaf weight for leaf j is derived from the objective function:

$$w^*j = -G_j / (H_j + \lambda)$$

$G_j = \sum g_i$ and $H_j = \sum h_i$ are the sums of gradients and Hessians for samples in leaf j . Unlike GBM which approximates leaf values using the mean of residuals, XGBoost solves for the optimal leaf weight directly from the objective function.

Split quality is assessed using a similarity score and gain formula. The similarity score for a node is:

$$\text{Similarity} = G^2 / (H + \lambda)$$

The gain from a split is:

$$\text{Gain} = \text{SimLeft} + \text{SimRight} - \text{SimParent} - \gamma$$

If $\text{gain} > \gamma$, the split is kept; otherwise it is pruned. This sophisticated pruning strategy controls tree complexity in XGBoost through the objective function.

XGBoost was implemented using `xgboost.XGBRegressor` with `n_estimators=200`, `learning_rate=0.1`, `max_depth=4`, `subsample=0.8`, `colsample_bytree=0.8`, `reg_lambda=1.5`, `reg_alpha=0.1`, and `objective='reg:squarederror'`. Both models used identical `n_estimators`, `learning_rate`, `max_depth`, and `subsample` to ensure a fair comparison isolating the effect of optimization order.

Results and Discussion

Sadly, for this demonstration both models achieved a nearly identical predictive performance. XGBoost demonstrating a marginal improvement in RMSE (10.825 vs 10.892 mmHg) and R^2 (0.594 vs 0.589). The similarity in predictive performance suggests that the data wasn't complex enough to justify using XGBoost for this task.

Model Performance on Test Set (n = 13,757)

Metric	Gradient Boosting (1st order)	XGBoost (2nd order)
RMSE (mmHg)	10.892	10.825
MAE (mmHg)	7.412	7.389
R^2 score	0.589	0.594
Training time (seconds)	3.6	0.1

The most interesting difference between the two models was speed. XGBoost trained 36 times faster than GBM using the same number of trees and identical settings. This comes from engineering optimizations built around the second-order framework: parallel split finding, pre-sorted data blocks, and cache-aware memory access (Chen & Guestrin, 2016). At production scale, retraining on millions of records daily, this translates directly to hours of compute time saved and reduced costs.

The practical significance of XGBoost's computational efficiency extends beyond this project. In production clinical systems retraining models daily on millions of patient records, a 36x reduction in training time translates directly to reduced cloud computing costs and faster model iteration cycles. At scale, the difference between first-order and second-order optimization could mean hours of compute time and cost savings.

References

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>

Di Cesare M, Perel P, Taylor S, Kabudula C, Bixby H, Gaziano TA, McGhie DV, Mwangi J, Pervan B, Narula J, Pineiro D, Pinto FJ. The Heart of the World. *Glob Heart*. 2024 Jan 25;19(1):11. doi: 10.5334/gh.1288. PMID: 38273998; PMCID: PMC10809869.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.

Sulianova. (2019). Cardiovascular Disease Dataset [Dataset]. Kaggle. <https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset>

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.